Heart Disease Prediction MLOps

End-to-end development, experiment tracking, serving, and operational readiness

Machine Learning Operations (MLOps) | Assignment 01

Dataset: UCI Heart Disease, Cleveland subset

Repository: github.com/adityathiyyagura/mlops-heart-disease-assignment

Executive summary

This project develops a reproducible binary classifier and wraps it in an API-oriented MLOps workflow. It covers source data acquisition, cleaning, exploratory analysis, model comparison, MLflow logging, automated tests, Docker packaging, Kubernetes configuration, request logging, and Prometheus metrics.

The evaluated models are Logistic Regression and Random Forest. Model selection uses mean ROC-AUC from five-fold stratified cross-validation on development data. A separate stratified 20% holdout is kept out of cross-validation and reported as a final test set.

Selected by tuned CV: Random Forest (mean CV ROC-AUC 0.897). Held-out test ROC-AUC: 0.932. These results are educational model metrics, not a clinical risk assessment or validation.

System at a glance

Data source -> cleaning/EDA -> stratified CV and held-out test
MLflow tracking -> selected fitted pipeline -> FastAPI
Docker image -> Kubernetes manifest | /metrics -> Prometheus scrape

Prepared from the project implementation and locally verified outputs. Deployment evidence is identified explicitly where it has not yet been captured.

1. Dataset acquisition and preparation

Data source and scope

The project uses the processed Cleveland Heart Disease dataset from the UCI Machine Learning Repository. It has 13 predictor columns and one disease target. The original target has values 0 through 4; this project maps 0 to no disease and any positive value to disease present.

The source is obtained by the download utility at scripts/download_data.py. The raw file is retained at data/heart.csv, while data/heart_cleaned.csv contains the complete-case data used for the deliverable. The cleaning path converts '?' to missing values and numeric-coerces feature columns.

Dataset counts

Measure	Observed value	Interpretation
Rows loaded for EDA	303	Target-valid source records before feature-row removal
Rows in cleaned CSV	297	Six records removed because feature values were missing
Columns	14	13 predictors plus binary target
Clean target 0	160	No disease in cleaned records
Clean target 1	137	Disease present in cleaned records
Missing values after cleaning	0	Verified in data/heart_cleaned.csv

Cleaning and leakage controls

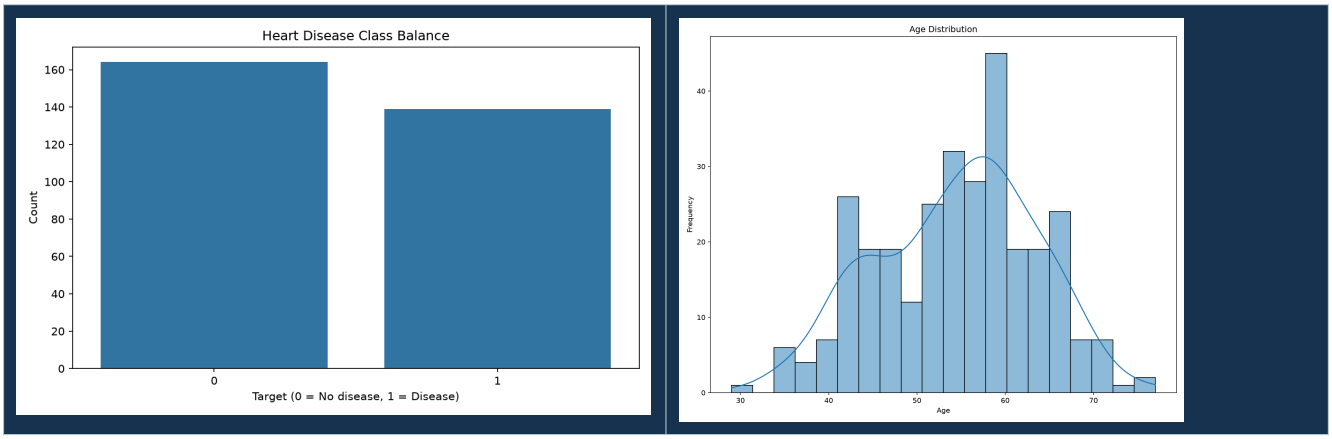
- The target is excluded from the input matrix and converted to a binary integer label.
- Rows with missing predictor values are removed before training; no test labels are used to fit the estimator.
- The train/test split is stratified to retain approximate class proportions in both partitions.
- Scaling is inside the Logistic Regression pipeline, so it is fitted independently within each CV fold and on development data for the final model.

This is complete-case handling rather than imputation. It is simple and reproducible for this small dataset, but may introduce bias if missingness is systematic.

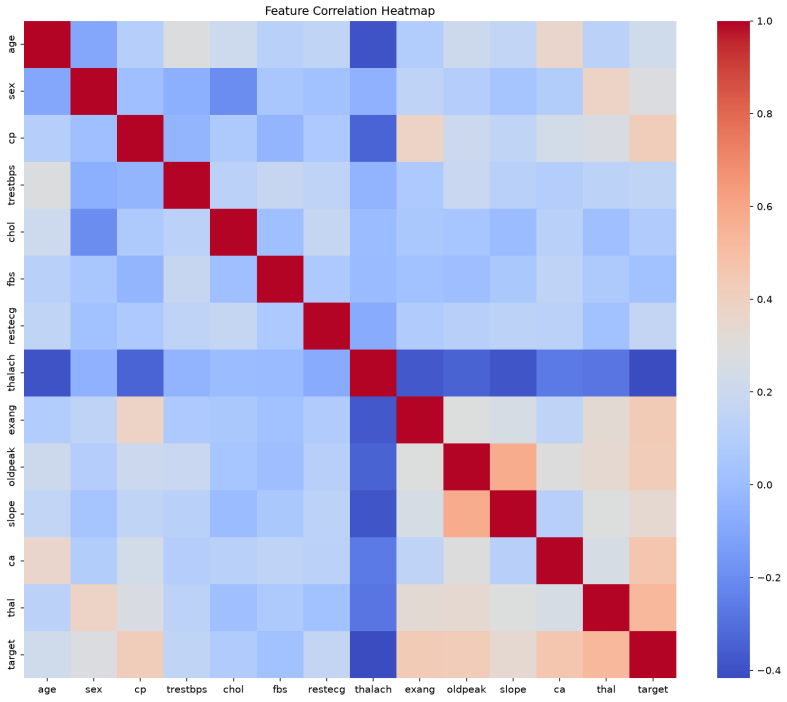
2. Exploratory data analysis

EDA is generated from the target-valid source data before feature rows with missing values are excluded. This is why EDA class counts (164 / 139) differ from cleaned modeling counts (160 / 137). The notebook and scripts save plots as artifacts for review.

Target distribution and age



Correlation overview



Interpretation

The target classes are moderately imbalanced rather than severely skewed. Age spans adult and older patients, with most records concentrated in middle-to-late adulthood. The correlation heatmap provides a first-pass view of linear associations; it is descriptive and does not establish causality or feature importance. Missing values in source predictors are handled after this descriptive stage for model preparation.

3. Features and model pipeline

The model uses all 13 input attributes. Nominal fields cp, restecg, and thal are one-hot encoded with unknown values ignored. The other ten fields remain numeric: Logistic Regression standardizes them, while Random Forest uses them unscaled.

Input schema

Feature group	Fields	Preparation
Demographic	age, sex	Numeric; source coding retained
Clinical measurements	trestbps, chol, thalach, oldpeak	Numeric coercion; complete-case rows
Nominal categories	cp, restecg, thal	OneHotEncoder(handle_unknown=ignore)
Binary/ordinal/numeric	sex, fbs, exang, slope, ca plus measurements	Retained numeric; scaled for Logistic Regression
Target	target	0 = absent; 1-4 mapped to present

Estimator construction

Candidate	Pipeline	Reproducibility controls
Logistic Regression	One-hot + StandardScaler -> LogisticRegression	Grid search C/class_weight; random_state=42
Random Forest	One-hot + passthrough -> RandomForestClassifier	Grid search tree size/depth/leaf; random_state=42

Inference uses the same named feature order as training. The API validates that all expected fields are present through its request schema. Model output is a binary class and the maximum class probability is returned as confidence; this confidence is not calibrated clinical probability.

The preprocessing is part of the persisted sklearn Pipeline, so training and inference share the same category encoding and scaling transformations.

4. Validation protocol and model results

A stratified 80/20 split is made once using random_state=42. Five-fold StratifiedKFold GridSearchCV (shuffle enabled, random_state=42) searches each model using only the 80% development partition and refits by ROC-AUC. The 20% holdout is reserved for independent final metrics and is not used for tuning.

Search and selected settings

Model	Grid	Selected settings
Logistic Regression	C: 0.1, 1, 10; class_weight: None/balanced	C=0.1; class_weight=None
Random Forest	n_estimators: 100/200; max_depth: None/6; min_samples_leaf: 1/2	200 trees; no depth cap; leaf=2

Cross-validation on development partition (mean +/- sample standard deviation)

Model	Accuracy	Precision	Recall	ROC-AUC
Logistic Regression	0.818 +/- 0.091	0.833 +/- 0.112	0.769 +/- 0.119	0.888 +/- 0.070
Random Forest	0.810 +/- 0.089	0.832 +/- 0.115	0.760 +/- 0.150	0.897 +/- 0.059

Final held-out test metrics

Model	Accuracy	Precision	Recall	ROC-AUC
Logistic Regression	0.850	0.880	0.786	0.951
Random Forest (CV-selected)	0.800	0.833	0.714	0.932

Selection and interpretation

Random Forest is selected by mean tuned CV ROC-AUC (0.897 versus 0.888). Logistic Regression scores higher on held-out ROC-AUC (0.951 versus 0.932), demonstrating why the holdout remains separate and results are reported independently rather than used for model selection.

The standard deviations indicate fold-to-fold variability on a small dataset. These point estimates have substantial uncertainty and are not claims of generalization to a different hospital, population, or contemporary cohort.

5. Experiment tracking and reproducibility

MLflow creates one run per model. Each run records tuned and estimator parameters, a serialized sklearn pipeline, held-out metrics, CV summary metrics, all three EDA plots, and held-out confusion matrix and ROC curve. CV metric names use cv__mean/std; holdout metrics use the held_out_ prefix.

Tracked CV summaries

Metric	Logistic Regression mean / std	Random Forest mean / std
Accuracy	0.818 / 0.091	0.810 / 0.089
Precision	0.833 / 0.112	0.832 / 0.115
Recall	0.769 / 0.119	0.760 / 0.150
ROC-AUC	0.888 / 0.070	0.897 / 0.059

Project artifacts and entry points

Path	Purpose
heart_disease_mlop/data.py	Download, load, and prepare dataset
heart_disease_mlop/modeling.py	Pipelines, CV, holdout evaluation, MLflow, model persistence
heart_disease_mlop/train.py	Command-line model training entry point
notebooks/	EDA and model comparison notebooks
artifacts/heart_disease_model.joblib	Persisted selected fitted pipeline
mlruns/	Local MLflow tracking data (runtime output)

A fixed random seed is used for splitting, CV shuffling, and estimators. For stronger production reproducibility, pin all dependency versions and record a dataset checksum; dependency ranges are currently specified in requirements.txt.

6. Prediction API and container

The FastAPI service exposes health and prediction routes. A request contains the 13 clinical predictor values; the response contains a binary prediction, a human-readable risk label, and a confidence score. The request schema rejects missing or malformed fields before model inference.

API surface

Route	Method	Purpose
/health	GET	Basic process health check
/predict	POST	Validate features and return class, label, confidence
/metrics	GET	Expose Prometheus text-format request counters and latency

Example request

```
{
  "age": 52, "sex": 1, "cp": 0, "trestbps": 125,
  "chol": 212, "fbs": 0, "restecg": 1, "thalach": 168,
  "exang": 0, "oldpeak": 1.0, "slope": 2, "ca": 2, "thal": 3
}
```

Local run commands

```
python -m heart_disease_mlop.train
docker build -t heart-disease-api:cv-tuned .
docker run --rm -p 8000:8000 heart-disease-api:cv-tuned
curl http://127.0.0.1:8000/health
```

The model is saved as a joblib pipeline and loaded by the service. The Dockerfile installs requirements, copies the project, exposes port 8000, and starts Uvicorn. A local TestClient prediction request returned HTTP 200 during verification; the confidence value is a model score, not a calibrated medical probability.

The API is an educational demonstration and must not be used to diagnose or make treatment decisions.

7. Testing and continuous integration

Automated tests cover the data interface, one-hot model preprocessing, model evaluation, and API prediction/metrics routes. The project workflow is triggered on pushes and pull requests to main/master.

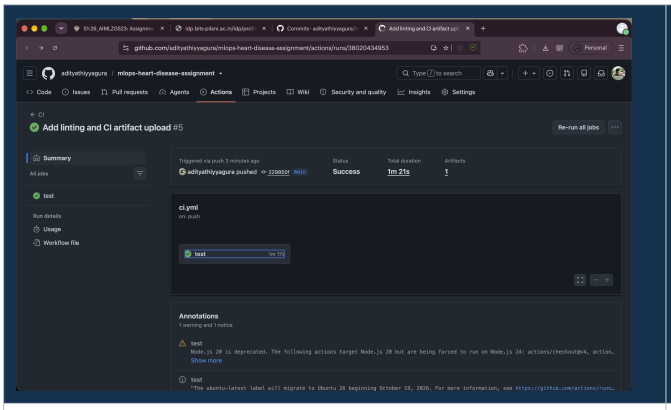
Current workflow stages

Stage	Action	Status in workflow
Checkout	Fetch repository source	Configured
Runtime	Set up Python 3.12	Configured
Dependencies	Install requirements.txt	Configured
Tests	Run pytest	Configured
Training	Run python -m heart_disease_mlop.train	Configured

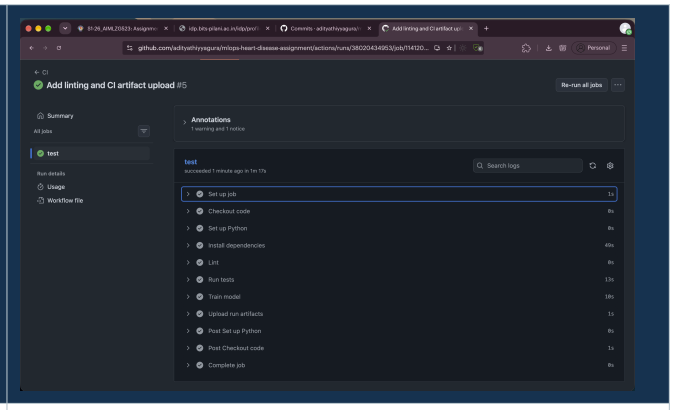
Quality gates and evidence

- Local verification after the encoding/tuning changes: 9 tests passed; Ruff passed. FastAPI lifecycle and TestClient deprecations remain non-failing warnings.
- Training was executed locally and produced the CV-selected Random Forest artifact and MLflow runs.
- GitHub Actions runs Ruff lint, pytest, and model training; it writes JUnit and training logs and uploads the model plus MLflow run data.
- CI screenshots show a previously successful hosted run; the new encoding/tuning changes must pass a fresh run after push. Screenshots are stored in screenshots/ci_run_summary.png and screenshots/ci_job_steps.png.

Hosted workflow evidence



Successful run summary



Lint, test, train, artifact upload

The uploaded workflow artifact preserves test results, the training log, the trained model, and MLflow tracking data for inspection.

8. Deployment and observability

Deployment configuration

The API was deployed to local Docker Desktop Kubernetes (v1.34.1, context docker-desktop). Both replicas reached Running/Ready. The LoadBalancer service is available at http://127.0.0.1:80 on this machine; there is no public cloud URL.

The updated model is packaged as heart-disease-api:cv-tuned and referenced by the Kubernetes manifest. Deployment is re-verified with kubectl rollout status and kubectl get pods/service. The Service exposes port 80 and targets container port 8000.

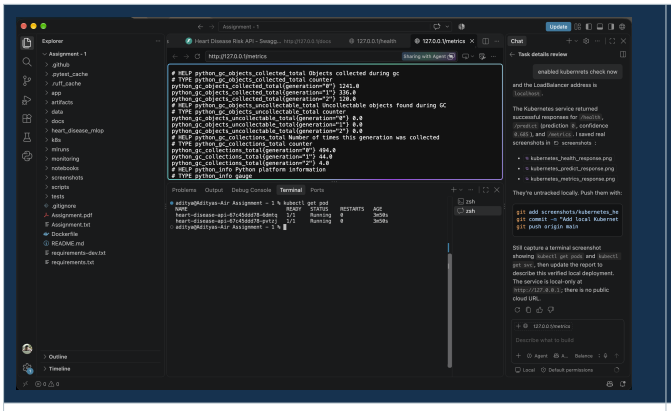
Request monitoring

Signal	Implementation	Use
Request count	http_requests_total labels method/path/status	Traffic and response status trends
Latency	http_request_duration_seconds histogram	Request duration distribution
Application log	Method, path, status, elapsed seconds	Local/container request diagnostics
Scrape config	monitoring/prometheus.yml	Prometheus target configuration

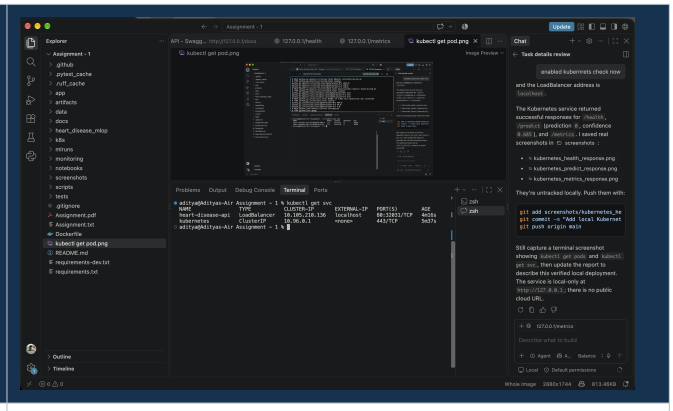
Observed locally

The updated Kubernetes LoadBalancer returned successful responses for /health, /predict, and /metrics. The sample prediction was class 0 with confidence 0.6524. Request logs were emitted by the API containers; metric counters are process-local to each replica.

Kubernetes pod and service screenshots



Two replicas Running/Ready



LoadBalancer on localhost:80

Endpoint screenshots are also included in screenshots/kubernetes_health_response.png, screenshots/kubernetes_predict_response.png, and screenshots/kubernetes_metrics_response.png.

9. Limitations, conclusion, and references

Limitations and responsible use

- Small, historical Cleveland subset: performance estimates may not transfer to another clinical setting or current population.
- Complete-case row removal can bias estimates if missingness is not random.
- Nominal cp, restecg, and thal fields are one-hot encoded; other binary and ordinal fields remain numeric and should be reviewed with domain expertise.
- The reported confidence is not probability calibration, and the model is not a diagnostic device.
- The held-out test set is one split; external validation and calibration analysis are not performed.

Conclusion

The implementation demonstrates a reproducible progression from public data to a tracked classifier, tested API, Docker image, verified local Kubernetes deployment, and request metrics. Five-fold stratified CV selects Random Forest by mean ROC-AUC while the final holdout remains separate. The deployment is local-only; there is no public cloud endpoint.

References

- UCI Machine Learning Repository, Heart Disease dataset: https://archive.ics.uci.edu/dataset/45/heart+disease
- Project repository: https://github.com/adityathiyyagura/mlops-heart-disease-assignment
- scikit-learn model evaluation and StratifiedKFold documentation: https://scikit-learn.org/stable/modules/cross_validation.html
- MLflow Tracking documentation: https://mlflow.org/docs/latest/ml/tracking/
- FastAPI documentation: https://fastapi.tiangolo.com/

Submission evidence checklist

Code, data, notebooks/scripts, tests, Dockerfile, requirements, CI workflow, report, Kubernetes manifest, and genuine CI/Docker/local-Kubernetes screenshots are included. Replace the generated slideshow with an actual pipeline recording before final submission.